

Reviewer Pack — Symmetric Group Fourier Coefficient Sum Exponential Gap Betw...

Entry e58723759ed4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 12:27:54 UTC

Symmetric Group Fourier Coefficient Sum Exponential Gap Between Permanent and Determinant

Entry ID: e58723759ed4 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-10 12:27:54 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory of Symmetric Groups **Field B** (complexity object): Boolean Circuit Complexity (Permanent vs Determinant)

Statement:

For any $n \geq 2$, the sum of absolute values of Fourier coefficients of the permanent function over S_n exceeds that of the determinant function by $\Omega(2^{\{n/2\}})$, where coefficients are computed using the Young symmetrizer decomposition of the group algebra.

Rationale (proposer’s reasoning):

The permanent’s positivity under symmetric group actions creates constructive interference in Fourier coefficients, while determinant’s alternating signs induce destructive interference. This gap mirrors the orbit closure separation in GCT, offering a combinatorial proxy for algebraic complexity.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 25a62a68592e5fa0

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def factorial(n):
    if n == 0:
        return 1
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def hook_length_formula(row):
    n = len(row)
    numerator = factorial(factorial(n))
    denominator = 1
    for cell in range(len(row)):
        row_length = row[cell]
        col_length = sum(1 for r in row if r > cell)
        denominator *= (row_length + col_length) * (cell + 1)
    return numerator // denominator

def young_symmetrizer(shape, sign):
    n = len(shape)
    basis_size = hook_length_formula(shape)
    basis = [[0] * basis_size for _ in range(basis_size)]

    def fill_basis():
        index = 0
        for perm in itertools.permutations(range(n)):
            if all(perm[i] <= shape[i] for i in range(n)):
                sign_factor = 1
                for cell in range(len(shape)):
                    row_length = shape[cell]
                    col_length = sum(1 for r in shape if r > cell)
                    sign_factor *= (-1) ** (sum(1 for p in perm[:cell+1] if p > cell))
                basis[index][index] += sign_factor
                index += 1

    fill_basis()
```

```

def multiply_matrix(A, B):
    result = [[0] * len(B[0]) for _ in range(len(A))]
    for i in range(len(A)):
        for j in range(len(B[0])):
            for k in range(len(B)):
                result[i][j] += A[i][k] * B[k][j]
    return result

def transpose_matrix(matrix):
    return [[matrix[j][i] for j in range(len(matrix))] for i in range(len(matrix[0]))]

def inverse_matrix(matrix):
    n = len(matrix)
    det = 0
    if n == 1:
        return [[1 / matrix[0][0]]]
    elif n == 2:
        det = matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0]
        return [[matrix[1][1] / det, -matrix[0][1] / det],
                [-matrix[1][0] / det, matrix[0][0] / det]]
    else:
        cofactors = []
        for i in range(n):
            minor = [row[:i] + row[i+1:] for row in matrix[1:]]
            cofactor = (-1) ** (i % 2) * determinant(minor)
            cofactors.append(cofactor)
        return transpose_matrix([cofactors[i:i+n] for i in range(0, len(cofactors), n)])

def determinant(matrix):
    if len(matrix) == 1:
        return matrix[0][0]
    elif len(matrix) == 2:
        return matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0]
    else:
        det = 0
        for i in range(len(matrix)):
            minor = [row[:i] + row[i+1:] for row in matrix[1:]]
            det += (-1) ** i * matrix[0][i] * determinant(minor)
        return det

def apply_symmetrizer(matrix):
    result = [[0] * len(matrix[0]) for _ in range(len(matrix))]
    for i in range(basis_size):
        for j in range(basis_size):
            if basis[i][j] != 0:
                result = add_matrices(result, multiply_matrix(multiply_matrix(inverse_matrix(basis[i]), matrix), basis[j]))
    return result

def add_matrices(A, B):
    return [[A[i][j] + B[i][j] for j in range(len(B[0]))] for i in range(len(B))]

return apply_symmetrizer(matrix)

def compute_fourier_coefficients(n):
    perm_basis = young_symmetrizer((n,), 1)

```

```

det_basis = young_symmetrizer((n,), -1)

def permanent(matrix):
    result = 0
    for perm in itertools.permutations(range(n)):
        sign = 1
        for i in range(n):
            sign *= matrix[i][perm[i]]
        result += sign
    return result

def determinant(matrix):
    det = 0
    for perm in itertools.permutations(range(n)):
        sign = (-1) ** sum(1 for p in perm if p > perm.index(p))
        det += sign * product([matrix[i][perm[i]] for i in range(n)])
    return det

def product(lst):
    result = 1
    for x in lst:
        result *= x
    return result

perm_coefficients = [permanent(perm_basis)]
det_coefficients = [determinant(det_basis)]

return sum(abs(coeff) for coeff in perm_coefficients), sum(abs(coeff) for coeff in det_coefficients)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_perm_sum = 0
    total_det_sum = 0
    instances_tested = 0

    for n in n_values:
        perm_sum, det_sum = compute_fourier_coefficients(n)
        total_perm_sum += perm_sum
        total_det_sum += det_sum
        instances_tested += 1

    ratio = total_perm_sum / total_det_sum
    conjecture_holds = ratio > math.exp(n / 4)
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "Fourier Coefficient Sum Ratio",
        "metric_value": ratio,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys

```

```

seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_ratio = sum(r["metric_value"] for r in results) / len(results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient data or support_fraction < 0.8")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2665b4a7.py", line 148, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2665b4a7.py", line 120, in run_trial
    perm_basis = young_symmetrizer((n,), 1)
                  ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2665b4a7.py", line 56, in young_symmetrizer
    for cell in range(len(row)):
                      ~~~~~
TypeError: object of type 'int' has no len()

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to TypeError in young_symmetrizer function, preventing data collection
 | next: Fix the young_symmetrizer function to handle integer inputs properly and re-run experiments

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	48801
2	preregistration	ollama_remote	qwen3:8b	0	0	25572
3	novelty	ollama_remote	qwen3:8b	0	0	20632
4	novelty	ollama_remote	qwen3:8b	0	0	14210
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19024
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18116
7	judge	ollama_remote	qwen3:8b	0	0	16786

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 163142 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/e58723759ed4.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e58723759ed4.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/e58723759ed4.tar.gz` (if generated)